

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

CODE:

Scenario 1

-- Create the Procedure

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest

IS

BEGIN

 UPDATE Accounts

 SET Balance = Balance + (Balance * 0.01)

 WHERE AccountType = 'Savings';

```
COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('Monthly interest processed successfully.');
```

```
END;
```

```
/
```

```
-- Execute the Procedure
```

```
BEGIN
```

```
    ProcessMonthlyInterest;
```

```
END;
```

```
/
```

```
-- Verify the Result
```

```
SELECT *
```

```
FROM Accounts;
```

OUTPUT:

Scenario 1

The screenshot displays an SQL IDE interface. On the left, the 'Navigator' pane shows a tree structure with 'My Schema' and 'Tables'. The 'Tables' section is expanded, listing 'ACCOUNTS', 'CUSTOMERS', 'EMPLOYEES', 'LOANS', and 'TRANSACTIONS'. The main editor pane shows a SQL script titled 'Exercise3_TransferFunds.sql'. The script contains the following code:

```
1 CREATE OR REPLACE PROCEDURE TransferFunds
2 (
3     p_fromAccount IN NUMBER,
4     p_toAccount IN NUMBER,
5     p_amount IN NUMBER
6 )
7 IS
8     v_balance NUMBER;
9 BEGIN
10
11     SELECT Balance
12     INTO v_balance
13     FROM Accounts
```

Below the editor, the 'Script output' tab is active, showing the execution results:

```
SQL> CREATE OR REPLACE PROCEDURE TransferFunds
(
  p_fromAccount IN NUMBER,
  p_toAccount IN NUMBER,...
Show more...

Procedure TRANSFERFUNDS compiled
Elapsed: 00:00:00.029
```

Query result Script output DBMS output Explain Plan SQL history					
  Download Execution time: 0.001 seconds					
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	1	Savings	4545	6/24/2026, 5:29:48
2	2	2	Checking	3000	6/24/2026, 5:29:48

Scenario 2

CODE:

-- Create the Procedure

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus

(

 p_department IN VARCHAR2,

 p_bonus IN NUMBER

)

IS

BEGIN

 UPDATE Employees

 SET Salary = Salary + (Salary * p_bonus / 100)

 WHERE Department = p_department;

 COMMIT;

 DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');

END;

/

-- Execute the Procedure

BEGIN

```
UpdateEmployeeBonus('IT', 10);  
  
END;  
  
/
```

-- Verify the Result

```
SELECT *  
  
FROM Employees;
```

OUTPUT:

Query result Script output DBMS output Explain Plan SQL history							
Download Execution time: 0.008 seconds							
	EMPLOYEEID	NAME	POSITION	SALARY	DEPARTMENT	HIREDATE	
1	1	Alice Johnson	Manager	77000	HR	6/15/2015, 12:00:00	
2	2	Bob Brown	Developer	66000	IT	3/20/2017, 12:00:00	

Scenario 3

CODE:

-- Create the Procedure

```
CREATE OR REPLACE PROCEDURE TransferFunds
```

```
(  
    p_fromAccount IN NUMBER,  
    p_toAccount IN NUMBER,  
    p_amount IN NUMBER  
)
```

```
IS
```

```
v_balance NUMBER;

BEGIN

-- Get the balance of the source account

SELECT Balance

INTO v_balance

FROM Accounts

WHERE AccountID = p_fromAccount;


-- Check if sufficient balance exists

IF v_balance >= p_amount THEN

    UPDATE Accounts

    SET Balance = Balance - p_amount

    WHERE AccountID = p_fromAccount;


    UPDATE Accounts

    SET Balance = Balance + p_amount

    WHERE AccountID = p_toAccount;


    COMMIT;


    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');
```

ELSE

```
    DBMS_OUTPUT.PUT_LINE('Insufficient Balance.');
```

END IF;

```
END;
```

/

-- Execute the Procedure

BEGIN

TransferFunds(1, 2, 500);

END;

/

-- Verify the Result

SELECT *

FROM Accounts;

OUTPUT:

Query result Script output DBMS output Explain Plan SQL history						
  Download ▾ Execution time: 0.001 seconds						
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED	
1		1	1 Savings	4045	6/24/2026, 5:29:48	
2		2	2 Checking	3500	6/24/2026, 5:29:48	